

Szent Benedek Technikum és Gimnázium Szegedi Tagintézménye

Nudli Fordító dokumentáció

Gyenes Béla, Jójárt Mátyás

TÉMAVEZETŐ:

BODROGI PÉTER RÓBERT

Szakképesítés neve:

Szoftverfejlesztő-és tesztelő (506131203)

Szeged, 2026. április 16.



Tartalomjegyzék

Ábrák jegyzéke1

1. Előszó	3
2. Felhasználói dokumentáció	4
2.1. Telepítés és indítás	4
2.1.1. Előfeltételek	4
2.1.2. Telepítési lépések	4
2.2. Környezeti változók beállítása	4
2.2.1. DeepL API	4
2.2.2. MongoDB csatlakozási string	5
2.2.3. Az alkalmazás indítása	8
2.3. A felhasználói felület ismertetése	8
2.3.1. Bejelentkezés és regisztráció	8
2.3.2. Dokumentáció és rólunk fül	9
2.3.3. Fordítás fülek	9
2.3.4. Profil	10
2.3.5. Előfizetés	11
2.3.6. Jelszó változtatása	12
2.3.7. Saját fiók törlése felhasználóként	12
2.3.8. Admin és moderátor panel	12
2.4. Támogatott nyelvek	14
3. Fejlesztői dokumentáció	15
3.1. Technikai stack	15
3.1.1. Frontend	15
3.1.2. Backend	15
3.1.3. Fejlesztési eszközök	15
3.2. Mappastruktúra	15
3.3. Frontend	16
3.3.1. App.jsx	16
3.3.2. UserInterface.jsx	16
3.3.3. Translate.jsx	17
3.3.4. MongoConnect.jsx	18
3.3.5. LanguageSelector.jsx	18
3.3.6. Egyéb Komponensek	19
3.4. Backend	19
3.4.1. Alapvető Konfiguráció	19
3.4.2. Végpontok	20
3.4.3. Fordítási végpontok	20
3.4.4. Admin Végpontok	20
3.4.5. Adatbázis séma	20
3.5. Fejlesztési leírás	21
3.5.1. Fejlesztési folyamat	21
3.5.2. Tesztelés	22
3.5.3. Build és deployment	23
3.6. Hibakezelés és megoldások	23
3.6.1. Általunk észlelt problémák ezelőtt	23
3.7. Jó, ha tudod...	24
3.8. Jövőbeli fejlesztési lehetőségek (roadmap)	24
4. Dokumentáció elkészítése	24
4.1. Mi az az Emacs Org Mode?	24
4.2. L ^A T _E X, avagy Latex	25
4.3. Org Mode és L ^A T _E X együtt használva	25

Ábrák jegyzéke

1.	Képen látható API oldal, kulcsok legenerálva.	5
2.	Hozzunk létre egy új projektet	5
3.	Nevezzük el projektünket	5
4.	Projektlétrehozás után hozzunk létre egy Cluster-t (adatbázist)	6
5.	Nevezzük el az adatbázisunk, válasszunk ki ingyenes szolgáltatót	6
6.	Hozzunk létre új felhasználót, majd kattintsunk a <i>Choose Connection Method</i> gombra.	7
7.	Itt látható a MongoDB csatlakozási string. Kattintsunk a gombra, hogy ki-másoljuk.	7
8.	Regisztráció	9
9.	Bejelentkezés	9
10.	Szövegfordítás	10
11.	Dokumentumfordítás	10
12.	Képfordítás	10
13.	"Profil" menüpont, valamint mentett fordítások	10
14.	Korlátlan mentésért és fordításokért szükségyszerű lesz előfizetnünk	11
15.	A program ellenőrzi, hogy az adatok validak-e. Ez csakis helyi, Stripe nélküli megoldás.	11
16.	A "Profil" menüpont alatt, a Fiók módosításában lehetőségünk van jelszavunk megváltoztatni	12
17.	A "Profil" menüpont alatt, a Fiók módosításában lehetőségünk van fiókunkat törölni	12
18.	Alap navbar, alap felhasználói jogosultságokkal	12
19.	Jogosulatlan belépés backend-re	13
20.	"Személyzeti" navbar, Backend menüponttal	13
21.	Adminok számára elérhető backend oldal	13
22.	Moderátorok számára elérhető backend oldal	14
23.	Üzenetek megtekintése személyzetként	14
24.	Sikeres tesztek regisztrációnál, bejelentkezésnél és fordításnál	22
25.	Ha valamiféle hiba történne teszteléskor, ezt látnánk.	22
26.	Az említett CORS hiba	23
27.	MongoDB "Hálózati hiba"	23
28.	Emacs logója	24
29.	Org Mode logója, a következő felirattal: " Szervezd az életed, <i>sima szövegben</i> "	25

1. Előszó

Applikációnk elkészítésének fő inspirációja egyszerűen azon a tényen alapult, hogy nem egy egyszerű webshopot, termék-rendelős weboldalt akartunk készíteni. Az ilyen típusú projektek bár hasznosak lehetnek tanulási célokra, sok esetben túlságosan sablonosak, és viszonylag gyorsan, kevés egyedi kihívással megvalósíthatók. Mi ezzel szemben olyan projektet szerettünk volna létrehozni, amely technikai szempontból érdekesebb, és több lehetőséget biztosít a különböző webfejlesztési eszközök és szolgáltatások alkalmazására.

Hosszabb ötletelési és tervezési folyamat előzte meg a végső döntést. Számos különböző ötlet merült fel: például egy fórum jellegű, 4chan-szerű oldal létrehozása, egy terminál stílusú felületre épülő webalkalmazás, illetve egy felhasználói fiókokkal működő blogrendszer. Mindegyik elképzelés hordozott magában érdekes lehetőségeket, végül olyan megoldást kerestünk, amely egyszerre hasznos, technológiailag releváns. Így választottuk a nyelvfordító webalkalmazás fejlesztését.

Inspirációnk elsősorban az ismertebb online fordítóoldalakról származott, mint például a DeepL, a Yandex, Google Fordító. Ezek a rendszerek mára a mindennapi internetes használat részévé váltak, mivel gyors és viszonylag pontos fordítási lehetőséget nyújtanak, gyakran több mint száz nyelven. Célunk az volt, hogy ezeknek a fordítók legfontosabb funkcióit egy saját, egyszerűen használható webes felületen leprogramozzuk, miközben megértjük és dokumentáljuk a mögöttük álló technológiai stackeket is.

A projekt alapját a DeepL API biztosítja, mely lehetővé teszi a szövegek fordítását 30 különböző nyelv között. Az API használatával a webalkalmazás képes valós időben feldolgozni a felhasználó által megadott szöveget, majd nagyon rövid időn belül vissza is adja annak lefordított változatát.

Az alkalmazás fejlesztése során külön figyelmet fordítottunk a felhasználói felület egyszerűségére és átláthatóságára. Célunk az volt, hogy a weboldal használata intuitív legyen, és a felhasználó gyorsan elérhesse a kívánt funkciókat. Bár nem UI/UX fejlesztők vagyunk, így is próbálkoztunk a felhasználóbarát felület kialakítására, így szintén híresebb fordítók designjáról inspirálódtunk.

Fejlesztők és felhasználók számára egyaránt ajánljuk a "Felhasználói útmutató" című fejezetet, mivel ez részletesen bemutatja a weboldal telepítésének folyamatát, a szükséges környezeti változók, csomagok, API-kulcsok beállítását. Ez a fejezet lépésről lépésre ismerteti azokat a lépéseket, amelyek szükségesek az alkalmazás működésbe helyezéséhez.

Összességében a projekt célja nem csupán egy működő fordítóalkalmazás létrehozása volt, hanem annak dokumentálása is, hogyan lehet modern webes technológiák (React, Vite, MongoDB) és külső szolgáltatások (DeepL) segítségével egy hasznos és gyakorlatban is alkalmazható rendszert megvalósítani.

2. Felhasználói dokumentáció

2.1. Telepítés és indítás

Az alkalmazás a Node.js és az npm csomagkezelő használatával telepíthető és indítható. A projekt a `tests/full-react-app` könyvtárban található (ezt később áttesszük a fő könyvtárba).

2.1.1. Előfeltételek

A projekt futtatásához szükséges:

- Node.js (verzió 18 vagy újabb)
- npm (Node Package Manager)
- MongoDB fiók (online vagy lokális MongoDB szerver)
- DeepL API kulcs (fordítási funkcióhoz)

2.1.2. Telepítési lépések

Menjünk a projekt könyvtárába (`tests/full-react-app`):

```
1 cd tests/full-react-app
```

Telepítsük az összes szükséges Node.js csomagot:

```
1 npm install
```

Hozzunk létre egy `.env` fájlt a projekt mappájában, és adjuk meg a szükséges változókat. Ezek létfontosságúak, mivel az alkalmazásunk ezeket a változókat fogja keresni pl. API hívásokra:

```
1 MONGO_URI=mongodb+srv://[ipafai_pap_felhasznalonev]:[fapipapipap_jelszo]@[cluster]
2 DB_NAME=appdb
3 PORT=3001
4 VITE_DEEPL_API_KEY=DeepL_API_Kulcsunk
```

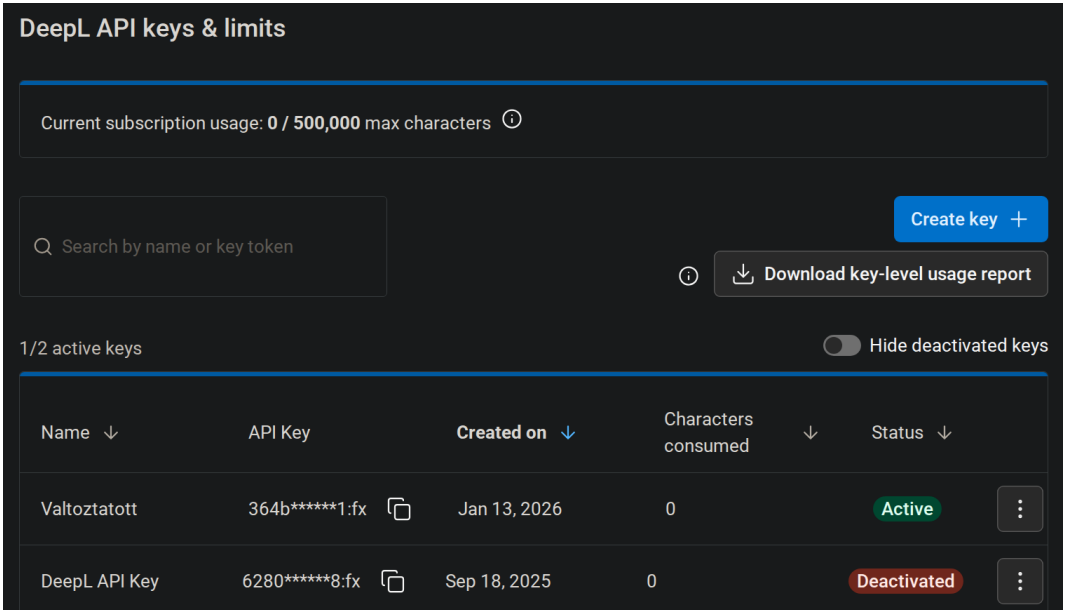
2.2. Környezeti változók beállítása

2.2.1. DeepL API

A program elindításához elengedhetetlen a környezeti változók beállítása. Weboldalunk alapértelmezetten a gyökérkönyvtárban található `.env` fájlból keresi a következő változókat: `MONGO\_URI`, `DB\_NAME`, `PORT` és `VITE\_DEEPL\_API\_KEY`. Valójában ezek közül csak kettőt, `MONGO\_URI` és `VITE\_DEEPL\_API\_KEY` változókat kell módosítanunk, a többi lényegében "hardcodeolt".

DeepL API kulcsunk legenerálásához:

1. Regisztráljunk a DeepL oldalára
2. Navigáljunk a DeepL API oldalára.
3. Kattintsunk a *Create key* gombra, majd hozzuk létre az API kulcsot. Ezután másoljuk ki, és illesszük be a `.env` fájlba.

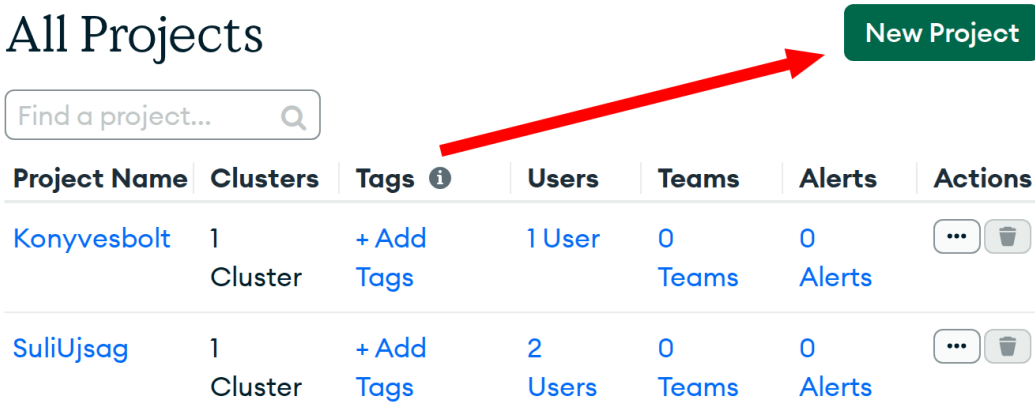


1. ábra. Képen látható API oldal, kulcsok legenerálva.

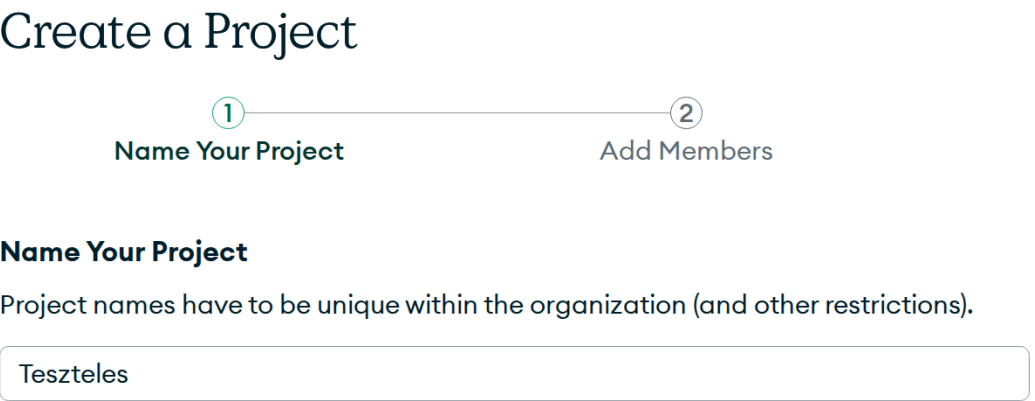
```
1 MONGO_URI='mongodb+srv://felhasznalonev:jelszo@cluster-mongodb-cime/'
2 DB_NAME=appdb
3 PORT=3001
4 VITE_DEEPL_API_KEY="DeepL API kulcsod"
```

2.2.2. MongoDB csatlakozási string

MongoDB adatbázisunkat egy bizonyos "csatlakozási string"-el tudjuk elérni. Navigáljunk a MongoDB oldalára, hozzunk létre új Cluster-t, majd másoljuk ki a csatlakozási linket. Lépések:

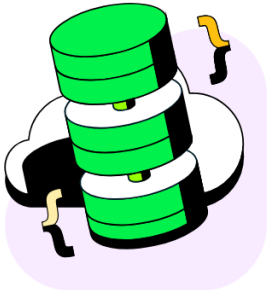


2. ábra. Hozzunk létre egy új projektet



3. ábra. Nevezzük el projektünket

Tesztelés Overview



Create a cluster

Choose your cloud provider, region, and specs.

+ Create

4. ábra. Projektlétrehozás után hozzunk létre egy Cluster-t (adatbázist)

M10

\$0.09/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE

10 GB

RAM

2 GB

vCPU

2 vCPUs

Flex

From \$0.011/hour
Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE

5 GB

RAM

Shared

vCPU

Shared

Free

For learning and exploring MongoDB in a cloud environment.

STORAGE

512 MB

RAM

Shared

vCPU

Shared

✔ Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name

You cannot change the name once the cluster is created.

Teszt

Provider

aws

Google Cloud

Azure

Quick setup

☒ Automate security setup ⓘ

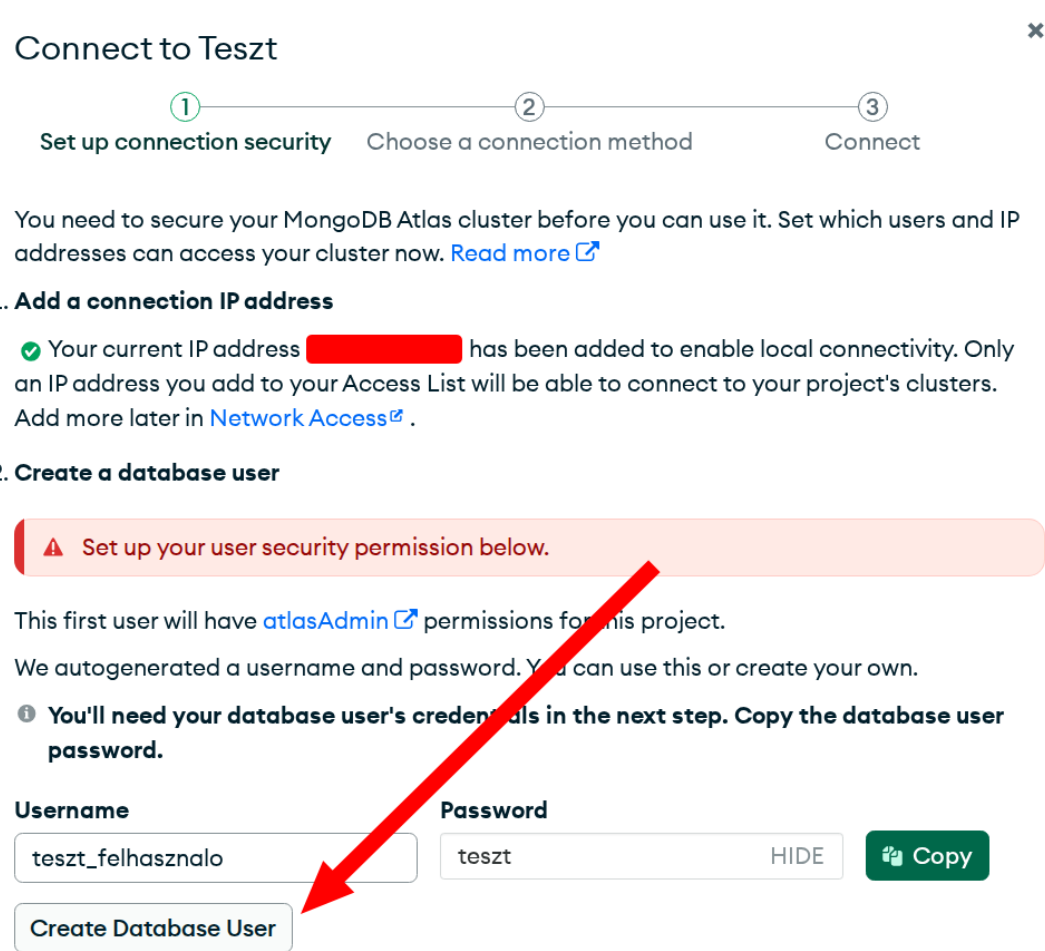
☐ Preload sample dataset ⓘ

5. ábra. Nevezzük el az adatbázisunk, válasszunk ki ingyenes szolgáltatót

Nudli Fordító

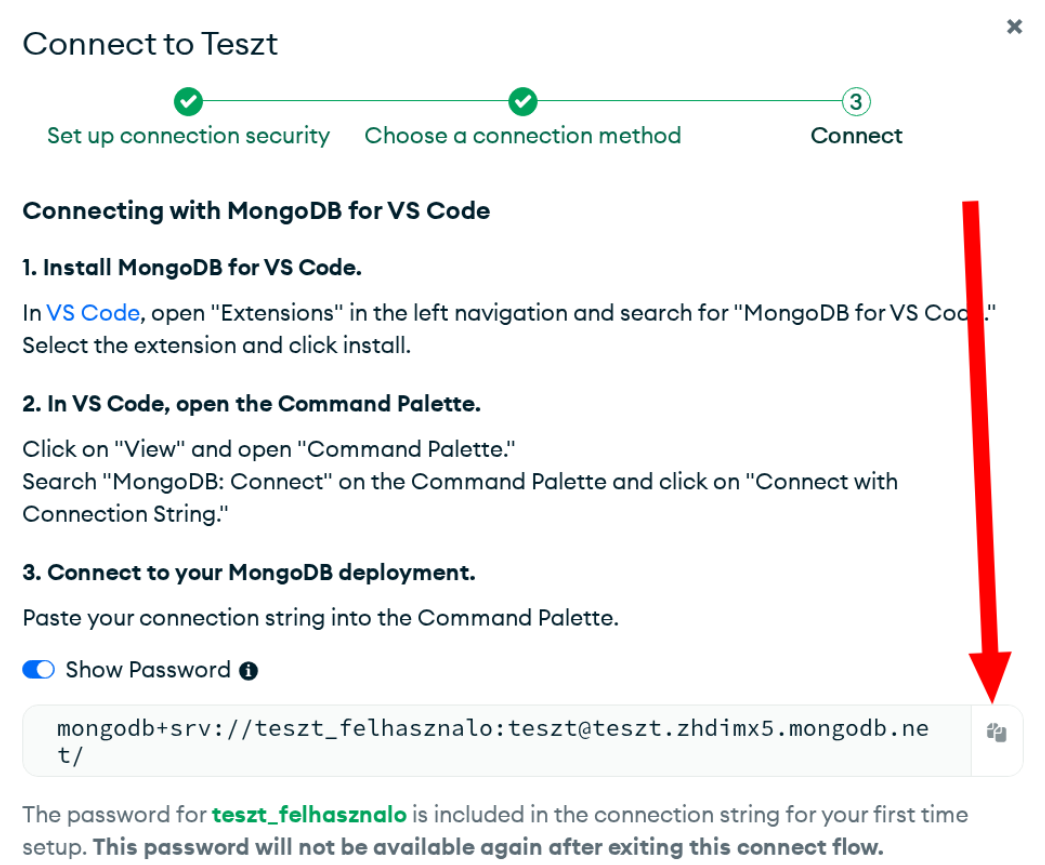
2026. április 16.

6



6. ábra. Hozzunk létre új felhasználót, majd kattintsunk a *Choose Connection Method* gombra.

Ezután nyomjunk rá a *MongoDB For VSCode* gombra. Itt láthatjuk a csatlakozási stringet:



7. ábra. Itt látható a MongoDB csatlakozási string. Kattintsunk a gombra, hogy kimásoljuk. Másoljuk ki, és adjuk meg a *MONGOURI* változónak, a *.env* fájlban.

Ha a továbbiakban csatlakozási problémákat észlelnénk, engedélyezzük a saját IP címünket, vagy egyszerűen a 0.0.0.0 IPv4 címet a Cluster beállításában. Az utóbb említett IP cím az összes létező IP cím számára hozzáférést ad az adatbázisunkhoz. **Jegyezzük meg, hogy ettől függetlenül jelszó kell, tehát lényegében nem teljesen feltörhető, de határo-**

zottan kijelenthető, hogy az adatbázis könnyebben feltörhető. Így a 0.0.0.0 IPv4 címet csak fejlesztési fázisban írjuk be.

2.2.3. Az alkalmazás indítása

Fejlesztési módban:

Két terminált kell megnyitnunk, hogy:

1. frontendet, valamint
2. backendet elindítsunk.

A frontend-, valamint backend szervereket egyben nem lehet elindítani-pipeolás kivételével, mert valójában a kettő külön létezik, így a frontend fejlesztőnek a backend szerver nem kell elindítani, és ugyanígy visszafelé is.

Frontend szerver elindításához adjuk ki ezt a parancsot:

```
npm run dev
```

Az alkalmazás a `http://localhost:5173` címen lesz elérhető.

Backend szerver indításához ezt:

```
npm run server
```

A backend szerver az `http://localhost:3001` porton fut.

Ha végleg leszeretnénk buildelni, esetleg külső szerverre helyezni a fordított fájlokat:

Build parancs:

```
npm run build
```

Ez lényegében már egy webszerveren is futható fájlokat ad ki a `public/` mappába.

Indítsuk el a Node.js szerveret, amely kiszolgálja a build-elt alkalmazást:

```
npm run start
```

2.3. A felhasználói felület ismertetése

Az alkalmazás egy navigációs sávval rendelkezik a tetején, amely különböző oldalakra irányítja a felhasználót.

2.3.1. Bejelentkezés és regisztráció

Ahhoz, hogy az alkalmazás teljes funkcionalitásait élvezhessük, bejelentkezésre van szükség. Ha még nem regisztráltunk volna, először azt tegyük meg:

Regisztráció:

1. Válasszuk ki a Regisztrációt, mivel még nincs fiókunk
2. Adjuk meg a felhasználónevünket és jelszavunkat (legalább 6 karakter)
3. Kattintsunk Regisztráció gombra
4. Sikeres regisztráció után automatikusan bejelentkeztet minket a weboldal

Regisztrálj a Nudlira

Felhasználónév

Jelszó

Mutat

Regisztrálok

Van már fiókod? [Bejelentkezés](#)

8. ábra. Regisztráció

Bejelentkezés:

- 1. Kattintsunk a Bejelentkezés gombra
- 2. Adjuk meg adatainkat
- 3. Kattintsunk Bejelentkezés gombra
- 4. Sikeres bejelentkezés után a fordítási oldalra dob minket a weboldal

Jelentkezz be a Nudli fordítóba

Felhasználónév

Jelszó

Mutat

Belépés

Nincs még fiókod? [Regisztráció](#)

9. ábra. Bejelentkezés

2.3.2. Dokumentáció és rólunk fül

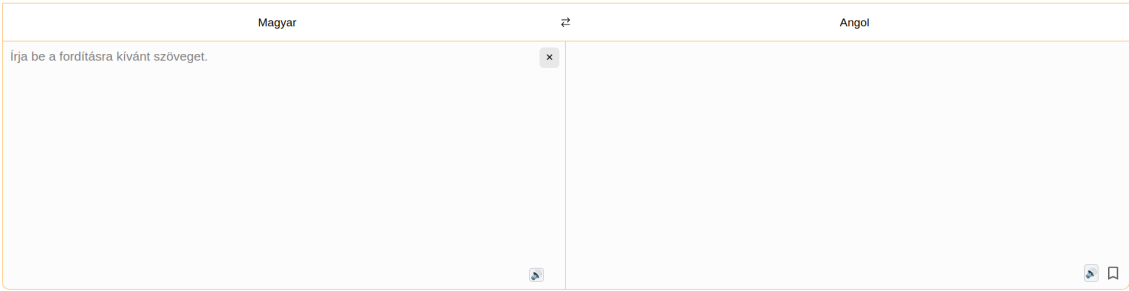
Az alkalmazás a következő további oldalakat kínálja:

- **Rólunk oldal (/about):** Az alkalmazás és a fejlesztőkről szóló információ.
- **Dokumentáció (/docs):** A dokumentáció, melyet most olvasol.

2.3.3. Fordítás fűlek

Három fordítási mód áll rendelkezésre:

- **Szöveg fordítása (Fordítás):** A legegyszerűbb fordítási mód. Válasszuk ki a forrás- és célnyelvet, írjuk be a szöveget a bal oldalon lévő mezőbe, és egy kis várakozás után a fordító automatikusan (pár miliszekundum után) lefordítja a beírt szöveget. A fordított szöveg a jobb oldalon jelenik meg, ugyanígy egy mezőben. Az alkalmazás támogatja a hangolvasást (text-to-speech) funkciót, ha azt a böngésző támogatja.



10. ábra. Szövegfordítás

- **Dokumentumfordítás (Dokumentumok):** Egyszerűbb dokumentumok fordítására alkalmas. Töltsünk fel PDF vagy Word dokumentumot, válasszuk a forrás- és célnyelvet, majd az alkalmazás lefordítja a teljes dokumentumot. Ezután lehetőségünk van letölteni a fordított dokumentumot, és így is tekinthetjük meg.



11. ábra. Dokumentumfordítás

- **Képfordítás (Képek):** Szöveg kinyeréséhez képekből OCR (Optical Character Recognition) technológiát alkalmazunk. A megismert szöveg majd lefordítódik a kiválasztott célnyelvre. Képet nem ad vissza, csupán csak szöveget.



12. ábra. Képfordítás

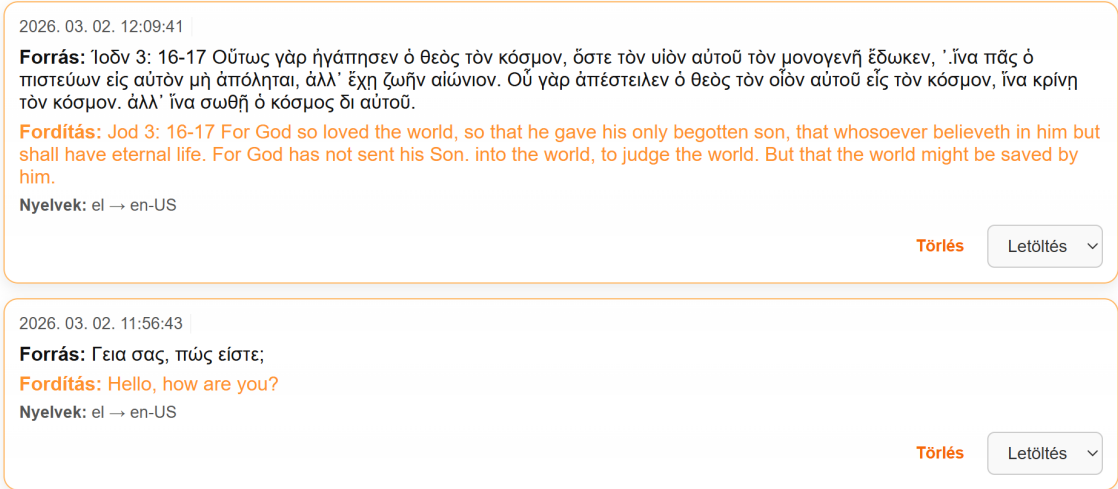
Lehetőségünk van a kedvenc fordításaink mentésére is, melyet a szövegfordításkor megjelenő könyvjelző ikonnal tehetünk. Azonban ez csak akkor működik, ha be vagyunk jelentkezve. A mentett fordításokat a "Profil" menüpont alatt találhatjuk.

2.3.4. Profil

Az Profil oldalon láthatjuk felhasználói tevékenységeinket:

- jelenlegi csomag,
- fordítási előzményeink (mentések)

Mentett fordítások



13. ábra. "Profil" menüpont, valamint mentett fordítások

Mentéseink száma nem korlátlan. Ahhoz, hogy végtelen mennyiségű szöveget tudjunk lementeni, elő kell fizetnünk az alkalmazásra. Ezt megtehetjük az "Előfizetés" menüpontban.

A felhasználó megteheti, hogy fiókját törli, fiókjának jelszavát megváltoztatja.

2.3.5. Előfizetés

Az alkalmazás ingyenes és fizetős csomagokat kínál:

Ingyenes csomag (Nudli):

- Limitált fordítások mentése (10 naponta)
- Limitált dokumentumfordítás (1 naponta)
- Limitált képfordítás (1 naponta)

Premium csomag (Nudli Plusz):

- Korlátlan fordítás mentése
- Korlátlan dokumentumfordítás
- Korlátlan képfordítás

Az előfizetéshez kattintsunk az Előfizetés oldalra és válasszuk ki a kívánt csomagot.

Előfizetés

Válassz csomagot a Nudli szolgáltatásaihoz

Nudli

Ft 0 HUF / hónap

A díjmentes csomag

- Szöveg fordítás
- 1 db kép és dokumentum fordítás naponta
- Max 10 db fordítás mentése

Jelenlegi csomagod

Díjmentes csomag.

Nudli Plusz

Ft 2,990 HUF / hónap

Továbbfejlesztett fordítási funkciók

- Fordítás mentése korlátlanul
- Korlátlan kép fordítás
- Korlátlan dokumentum fordítás

Bővítés – Nudli Plusz-ra

Előfizetés havi díjjal.

14. ábra. Korlátlan mentésért és fordításokért szükségszerű lesz előfizetnünk

Ha ténylegesen előszeretnénk fizetni, a "Bővítés" gombra kattintva megtehetjük. Ez egy új menüpontot hoz elő, ahová személyes adataink beírását követően fel is regisztrálhatunk az előfizetésre:

PNG

WEBP

JPG

Tölts fel vagy húzz ide fájlokat a fordításhoz.

Fájlok böngészése

15. ábra. A program ellenőrzi, hogy az adatok validak-e. Ez csakis helyi, Stripe nélküli megoldás.

Nudli Fordító

2026. április 16.

11

2.3.6. Jelszó változtatása

Jelszó módosítása

Jelenlegi jelszó

Új jelszó

Új jelszó megerősítése

Mentés

16. ábra. A "Profil" menüpont alatt, a Fiók módosításában lehetőségünk van jelszavunk megváltoztatni

2.3.7. Saját fiók törlése felhasználóként

Fiók törlése

Biztosan töröld a fiókot? Ez visszafordíthatatlan. A folytatáshoz írd be a jelszavad.

Törlés

17. ábra. A "Profil" menüpont alatt, a Fiók módosításában lehetőségünk van fiókunkat törölni

2.3.8. Admin és moderátor panel

Admin és moderátorok számára elérhető egy külön panel, ahol kezelhetik a felhasználókat. A weboldal adminjainak lehetősége van a felhasználók fiókjának törlésére, jelszavuk megváltoztatására, valamint admin jogok adására, elvételére. Moderátoroknak lehetőségük van a felhasználói üzenetek moderálására, tehát törlésére.

Erre a panelra a "Backend" gombbal juthatunk. Ha a felhasználónak jogosultsága van eme panel elérésére, ez a navbar-ban fog megjelenni.



18. ábra. Alap navbar, alap felhasználói jogosultságokkal

Ez az alapvető navbar, ha valaki már regisztrált és bejelentkezett oldalunkra. Mivel nem moderátor, és nem is admin, így "Backend" menüpont nem érhető el. Ha ezt valaki megpróbálná kikerülni, és a /backend címre navigálna, jogosulatlan lenne a belépésre, amit itt láthatunk is:



19. ábra. Jogosulatlan belépés backend-re

Most nézzünk példát arra, hogy néz ki egy moderátornak vagy adminnak a navbar:



20. ábra. "Személyzeti" navbar, Backend menüponttal

Ha adminként lépünk a "Backend" menüpontra, az admin panelra navigálhatunk:

Admin panel

Keresés...

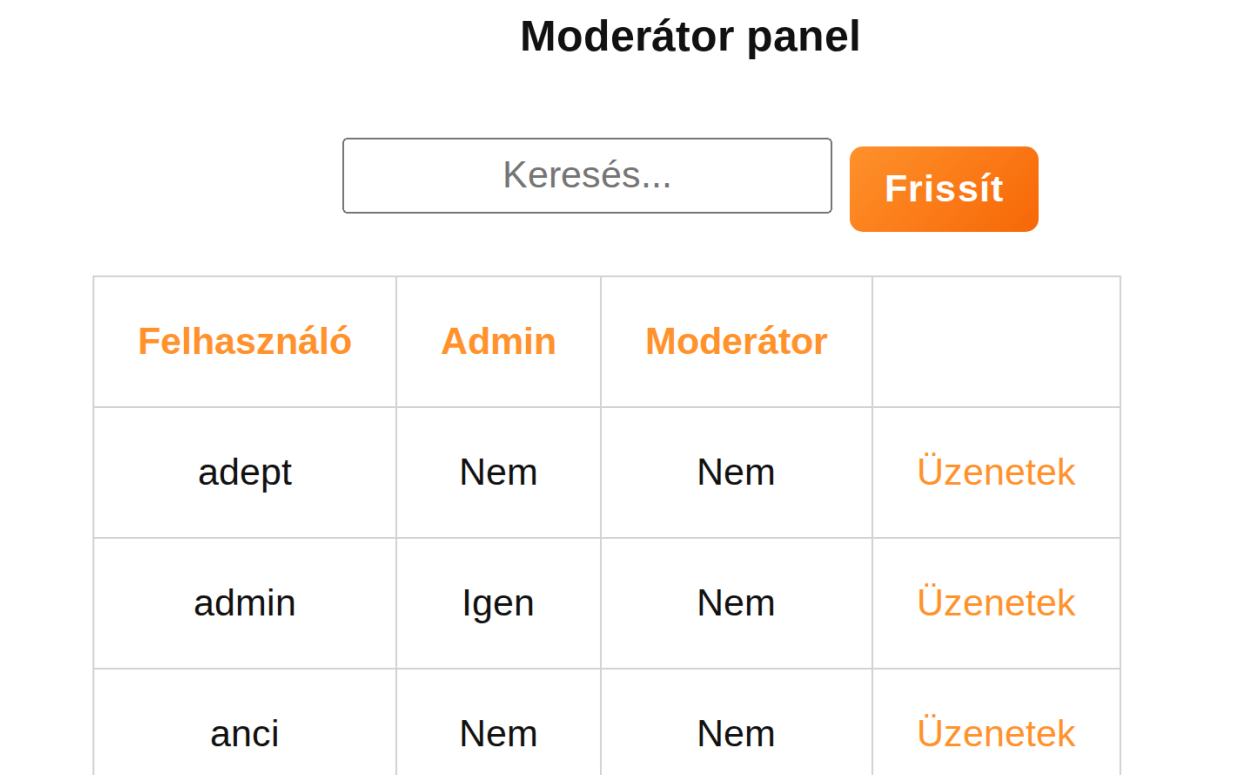
Frissít

Felhasználó	Admin	Moderátor					
adept	Nem	Nem	Üzenetek	Admin ad	Mod ad	Jelszó vált.	Fiók törlése
admin	Igen	Nem	Üzenetek	Admin elvesz	Mod ad	Jelszó vált.	Fiók törlése
anci	Nem	Nem	Üzenetek	Admin ad	Mod ad	Jelszó vált.	Fiók törlése
anna	Nem	Nem	Üzenetek	Admin ad	Mod ad	Jelszó vált.	Fiók törlése
apex565	Nem	Igen	Üzenetek	Admin ad	Mod eltávolít	Jelszó vált.	Fiók törlése

21. ábra. Adminok számára elérhető backend oldal

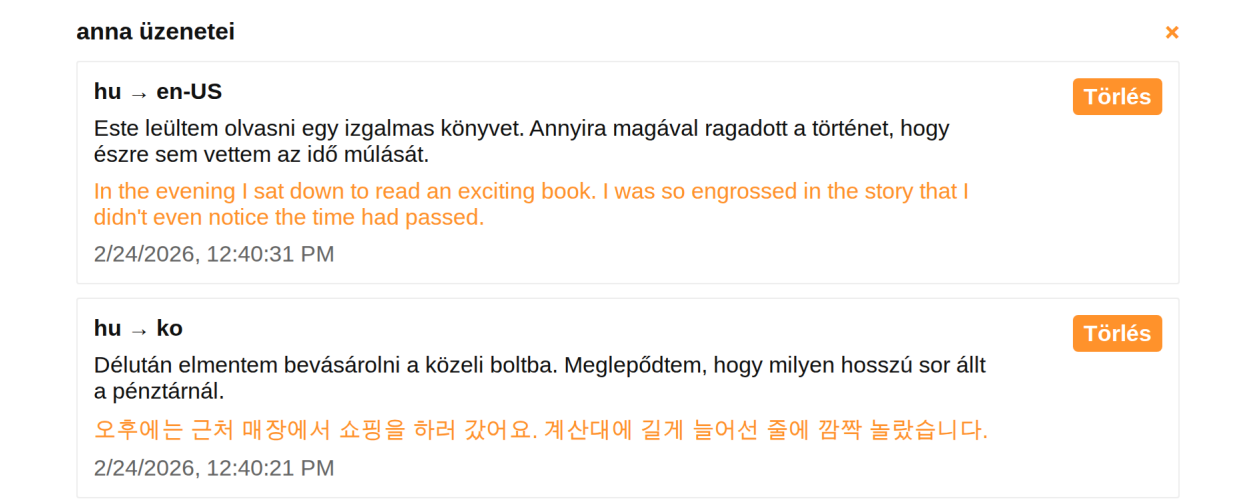
Ahogy látható, az alkalmazás adminjai képesek felhasználók üzeneteinek megtekintésére; admin jogok, moderátori jogok elvételére/adására, jelszó változtatására, valamint a fiók törlésére.

Azonban ha moderátorként lépünk a "Backend" menüpontra, a moderátor panelra ugrunk, amely teljesen máshogy néz ki:



22. ábra. Moderátorok számára elérhető backend oldal

A moderátor panel különbözik attól, amelyet az admin panelban láthattunk. Az alkalmazás moderátorai csak üzeneteket tudnak megtekinteni, és ezeket tudják lényegében moderálni. A nem helyénvaló üzenetek tudják törölni.



23. ábra. Üzenetek megtekintése személyzetként

Az "Üzenetek" gombra kattintva ez a modál ugrik elő. Így néz ki a kiválasztott felhasználó üzenetei.

2.4. Támogatott nyelvek

A Nudli fordító a következő nyelveket támogatja fordításhoz:

- arab, bolgár, cseh, dán, német, görög, angol, spanyol, észt, finn, francia, magyar, indonéz, olasz, japán, koreai, litván, lett, norvég, holland, lengyel, portugál, román, orosz, szlovák, szlovén, svéd, török, ukrán, vietnami, kínai (egyszerűsített és tradicionális)

A fordítás a DeepL fordítási API-n keresztül történik.

3. Fejlesztői dokumentáció

3.1. Technikai stack

Lentebb említett csomagok verziószámára kíváncsi fejlesztőknek: szíveskedjetek a `package.json` fájlban böngészni, mert a projekt frissítésekor gyakori, hogy a csomagok is frissülnek.

3.1.1. Frontend

- **React** - Komponensek, RESTful architektúra
- **React Router DOM** - Szerver oldali navigáció (SPA routing)
- **SCSS/Sass** - Stílusozás és CSS preprocessor
- **React Select** - Csinos legördülő menü komponens
- **React Icons** - SVG ikonok
- **Tesseract.js** - OCR feldolgozáshoz képekből
- **Vite** - Build eszköz, valamint dev szerver

3.1.2. Backend

- **Express.js** - HTTP szerver keretrendszer
- **MongoDB** - NoSQL adatbázise
- **DeepL Node** - API kliens
- **Dropzone** - Dokumentum-és képfeltöltés szépítéséhez
- **CORS** - Cross-origin kérések kezeléséhez
- **dotenv** - Környezeti változók kezeléséhez

3.1.3. Fejlesztési eszközök

- **Jest 30.2.0** - Unit teszteléshez
- **Nodemon 3.1.10** - Szerver automatikus újraindítás fejlesztéskor

3.2. Mappastruktúra

A projekt fő mappájának szerkezete:

```
tests/full-react-app/
|-- public/                # Statikus fájlok (irreleváns)
|-- src/
|   |-- components/        # React komponensek
|   |   |-- TranslateText.jsx    # Szöveg fordító
|   |   |-- TranslateDocs.jsx    # Dokumentum fordító
|   |   |-- TranslateImg.jsx     # Kép fordító (OCR)
|   |   |-- LanguageSelector.jsx # Nyelvválasztás
|   |   |-- MongoConnect.jsx     # Fontos API kérések
|   |   |-- UserInterface.jsx    # Navigáció
|   |   |-- Profile.jsx          # Felhasználó profil
|   |   |-- Subscription.jsx     # Előfizetési oldal
|   |   |-- About.jsx            # Rólunk oldal
|   |   |-- Documentation.jsx    # Dokumentáció (pdf)
|   |   |-- Footer.jsx           # Footer
|   |   |-- __tests__/          # Unit tesztelések
|   |   |-- *.scss              # Komponens stílusok SASS-al
|   |-- main.jsx              # Fő React fájl
|   |-- App.jsx                # Fő App komponens és Routeolás
|   |-- langs.json             # Támogatott nyelvek lista
|   |-- index.css
```



```
|-- server.js           # Express backend szerver
|-- vite.config.js      # Vite konfigurációs fájl
|-- jest.config.js      # Jest tesztelési konfiguráció
|-- package.json        # npm függőségek
|-- .env                # Környezeti változók
|-- index.html          # Fő HTML fájl (SPA erre megy)
```

3.3. Frontend

3.3.1. App.jsx

A fő alkalmazás komponens, amely az Router struktúrát és a tartalom elrendezését kezeli.

Fő funkciók:

- React Router konfiguráció
- Egyéb routingok
- Fordítási oldalak (szöveg, dokumentum, kép)
- Admin és profil oldalak
- 404 oldal kezelés

Alább a komponensek, stílusfájlok importálását láthatjuk:

```
1 // Stílusozás
2 import './components/Site.scss'
3 import './components/ColorScheme.scss'
4
5 // Komponensek
6 import Fordito from './components/TranslateText';
7 ...
8 import MongoConnect from './components/MongoConnect';
9 import ModeSelect from './components/ModeSelect';
10 import SubFooter from './components/SubFooter';
11 import Introduction from './components/Introduction';
12 import PageNotFound from './components/PageNotFound';
13 import Subscription from './components/Subscription';
```

Most pedig

```
1 <Routes>
2   <Route path="*" element={<PageNotFound />} />
3   <Route path="/login" element={<MongoConnect />} />
4   <Route path="/" element={<Navigate to="/translate" replace />} />
5
6   <Route path="/backend" element={<MongoConnect />} />
7   <Route path="/admin" element={<Navigate to="/backend" replace />} />
8   <Route path="/translate" element={<> <Introduction /> <ModeSelect /><
Outlet /> <SubFooter /></>>
9     <Route index element={<Fordito user={user} />} />
10    <Route path="docs" element={<TranslateDocs user={user} />} />
11    <Route path="img" element={<TranslateImg user={user} />} />
12  </Route>
13
14  <Route path="/about" element={<About />} />
15  <Route path="/profile" element={<Profile />} />
16  <Route path="/subscription" element={user ? <Subscription /> : <
Navigate to="/login" replace />} />
17 </Routes>
```

3.3.2. UserInterface.jsx

A navigációs sáv komponens az alkalmazás tetején. Ez kezeli a:

- oldal navigációs linkjeit,
- felhasználó bejelentkezését, illetve kijelentkezését,
- témaválasztást (ez lehet sötét, világos és automatikus téma),

- mobiltelefon esetén hamburger menü.

```
1 {user?.isAdmin && (
2   <Link to="/backend" onClick={() => setMobileOpen(false)}>Backend</Link>
3 )}
4 {user?.isModerator && (
5   <Link to="/backend" onClick={() => setMobileOpen(false)}>Backend</Link>
6 )}
7 {user ? (
8 <Link to="/subscription" onClick={() => setMobileOpen(false)}>Elofizetes</Link>
9 ) : null}
10 <Link to="/about" onClick={() => setMobileOpen(false)}>Rólunk</Link>
11 <a href="/docs/dokumentacio.pdf" target="_blank" onClick={() =>
    setMobileOpen(false)}>Dokumentacio</a>
12 {user ? (
13 <>
14   <Link to="/profile" onClick={() => setMobileOpen(false)}>{'Profil'}</Link>
15
16 </>
17 ) : (
18 <Link to="/login" onClick={() => setMobileOpen(false)}>Bejelentkezés</Link>
19 >
20 )}
```

3.3.3. Translate.jsx

A Translate.jsx három fő részből áll. Ez a szövegfordítás, képfordítás és dokumentum fordítás.

Szövegfordítási:

- forrás és célnyelv kiválasztása, ezután
- a felhasználó beír valamilyen szöveget a szövegmezőbe
- a komponens POST kérést küld a backend /translate végpontjára,
- DeepL API feldolgozza a fordítást,
- a felhasználó megkapja a lefordított szöveget

Képfordítás: Kép feldolgozó és OCR komponens. A Tesseract.js könyvtárra támaszkodik, első lépésként. Következésképpen működik:

- kép feltöltés vagy kiválasztás
- Tesseract.js OCR-re feldolgozza a képet -> szöveget nyer ki, majd ezt a szöveget átadja a DeepL API-nak,
- DeepL fordítja,
- a felhasználó megkapja a lefordított szöveget

Dokumentumfordítás: Dropzone segítségével kezeli a fájlfeltöltést, csak úgy, mint a kép-feltöltésnél. A dropzone drag and drop támogatásban segít.

```
1 ...
2 const {
3   sourceLang,
4   targetLang,
5   setSourceLang,
6   setTargetLang,
7   sourceOptions,
8   targetOptions,
9   swapLanguages,
10 } = useLanguageSelector({ initialSource: 'hu', initialTarget: 'en-US' })
11 ...
```

3.3.4. MongoConnect.jsx

Felhasználói bejelentkeztetés, admin felület. Ezen kívül:

- bejelentkezés,
- regisztráció,
- admin felület,
- jelszóváltoztatás,
- admin jogok kezelése.

Egy részlet a fájlból, mely admin-vagy moderátorként lekéri a különböző felhasználók mentett fordításait.

```
1 async function fetchTranslations(userFilter) {
2   const path = '/api/translations' + (userFilter ? '?user=' +
    encodeURIComponent(userFilter) : '');
3   const r = await api(path, 'GET', null, false);
4   if (!r.ok) return setAdminMsg(r.msg || 'Hiba a fordítások betöltésénél');
    ;
5   const grouped = {};
6   (r.translations || []).forEach(t => {
7     if (!grouped[t.userName]) grouped[t.userName] = [];
8     grouped[t.userName].push(t);
9   });
10  setTranslationsByUser(grouped);
11 }
```

3.3.5. LanguageSelector.jsx

Újrafelhasználható React hook komponens nyelvválasztás kezeléséhez. Hook funkciók:

- Forrás és cél nyelvválasztás
- Nyelvek felcserélése
- Alap nyelvek beállítása

Az alábbi kódsorokban látható, hogy a nyelvválasztó program a langs.json fájlból keresi a nyelveket.

```
1 import { languages_target, languages_source } from "../langs.json";
2
3 ...
4
5 export function useLanguageSelector({ initialSource = 'hu', initialTarget
    = 'en-US' } = {}) {
6   const [sourceOptions] = useState(languages_source);
7   const [targetOptions] = useState(languages_target);
8
9   const [sourceLang, setSourceLang] = useState(
10    sourceOptions.find(l => l.code === initialSource) || sourceOptions[0] ||
        null
11  );
12   const [targetLang, setTargetLang] = useState(
13    targetOptions.find(l => l.code === initialTarget) || targetOptions[0] ||
        null
14  );
15
16  ...
```

Az előbb említett langs.json alább látható:

```
1 {
2   "languages_source": [
3     ...
4     {"name": "Szloven", "code": "sl"},
5     {"name": "Sved", "code": "sv"},
6     {"name": "Torok", "code": "tr"},
7     {"name": "Ukran", "code": "uk"},
8     {"name": "Kinai (egyszerusiatett)", "code": "zh"},
```

```
9   {"name": "Kinai (hagyományos)", "code": "zh-HANT"}
10 ],
11
12   "languages_target": [
13     {"name": "Arab", "code": "ar"},
14     {"name": "Bolgar", "code": "bg"},
15     {"name": "Cseh", "code": "cs"},
16     {"name": "Dan", "code": "da"},
17     {"name": "Nemet", "code": "de"},
18     {"name": "Gorog", "code": "el"},
19     {"name": "Angol", "code": "en-US"},
20 ...
21 ]
22 }
```

3.3.6. Egyéb Komponensek

- **Introduction.jsx**: Üdvözlő szöveg
- **ModeSelect.jsx**: Módválasztó
- **Subscription.jsx**: Előfizetési lap
- **Profile.jsx**: Felhasználói profil
- **About.jsx**: Fejlesztőkről
- **Documentation.jsx**: Dokumentáció, melyet most olvas a kedves felhasználó
- **Footer.jsx**: Egyéb információk, lábjegyzék, stb.

3.4. Backend

3.4.1. Alapvető Konfiguráció

A backend Express.js szerveret a `server.js` fájlban találjuk:

```
1 import express from "express";
2 import cors from "cors";
3 import { MongoClient } from "mongodb";
4 import dotenv from "dotenv";
5
6 dotenv.config();
7 const app = express();
8 app.use(cors());
9 app.use(express.json());
10 const client = new MongoClient(process.env.MONGO_URI);
11 await client.connect();
12 const db = client.db(process.env.DB_NAME || "appdb");
```

A mentett fordítások TXT, PNG vagy JPG formátumban való letöltésének kódját a `downloadUtils.js` fájlban találhatjuk.

```
1 export function downloadAsTXT(item, username) {
2   const content = `${new Date(item.createdAt).toLocaleString()}
3   Forras: ${item.text}
4   Forditas: ${item.translation}
5   Nyelvek: ${item.source_lang} -> ${item.target_lang}`;
6   // use explicit UTF-8 charset so non-Latin characters are preserved
7   const blob = new Blob([content], { type: 'text/plain;charset=utf-8' });
8   const url = URL.createObjectURL(blob);
9   const a = document.createElement('a');
10  a.href = url;
11  a.download = `${new Date(item.createdAt).toLocaleDateString()}_${username}_forditas.txt`;
12  document.body.appendChild(a);
13  a.click();
14  document.body.removeChild(a);
15  URL.revokeObjectURL(url);
16 }
```

3.4.2. Végpontok

POST /api/register: Új felhasználó regisztrálása.

JSON így néz ki:

```
1 {  
2   "username": "felhasznalonev",  
3   "password": "jelszo"  
4 }
```

POST /api/login: Felhasználó bejelentkeztetése.

Ha a bejelentkeztetés sikeres, ezt JSON-body választ kapjuk:

```
1 {  
2   "ok": true,  
3   "msg": "Sikeres bejelentkezés",  
4   "username": "felhasznalonev",  
5   "isAdmin": false,  
6   "plan": {"id": "free", "name": "Nudli"}  
7 }
```

3.4.3. Fordítási végpontok

POST /translate: Szöveg fordítása DeepL API-n keresztül.

JSON kérés:

```
1 {  
2   "text": "Forditando szoveg",  
3   "source_lang": "hu",  
4   "target_lang": "en-US",  
5   "userName": "felhasznalonev"  
6 }
```

Válasz, ha sikeres:

```
1 "translation": "Your text has been translated!"
```

3.4.4. Admin Végpontok

1. GET /api/users Összes felhasználó lekérdezése (csak admin számára). Admin header a kéréskor: x-admin:1
2. PATCH api/users:username/password Felhasználó jelszavának megváltoztatása (csak admin).
3. PATCH api/users:username/toggle-admin Admin jogok be/kikapcsolása (csak admin).

3.4.5. Adatbázis séma

Az UML diagramok olyan rajzok, amelyek segítenek megérteni, hogyan épül fel egy program vagy rendszer. Megmutatják, milyen részekből áll a szoftver, és ezek a részek hogyan kapcsolódnak egymáshoz. Például ábrázolhatják az adatbázis tábláit, a bennük lévő adatokat és a köztük lévő kapcsolatokat. Az ilyen diagramok segítenek abban, hogy a fejlesztők és a nem fejlesztők is könnyebben átlássák a rendszer működését. Gyakran használják tervezéshez, dokumentációhoz és hibák átlátásához is.

users

_id: ObjectId

username: string

usernameLower: string

password: string

admin: boolean

isAdmin: boolean

isModerator: boolean

imagesCount: int

imagesCountDate: date

docsCount: int

docsCountDate: date

planId: string / null

subscribed: boolean

translations

_id: ObjectId

username: string

usernameLower: string

password: string

admin: boolean

isAdmin: boolean

isModerator: boolean

imagesCount: int

imagesCountDate: date

docsCount: int

docsCountDate: date

planId: string / null

subscribed: boolean

1. users Kollekcio

```
1 {
2   _id: ObjectId,
3   username: String (unique),
4   password: String,
5   isAdmin: Boolean,
6   subscribed: Boolean,
7   planId: String,
8   createdAt: Date
9 }
```

2. translations Kollekcio

```
1 {
2   _id: ObjectId,
3   username: String,
4   text: String,
5   translation: String,
6   source_lang: String,
7   target_lang: String,
8   type: String,
9   timestamp: Date
10 }
```

Ezeket objektumokkent felfoghatjuk, es minden egyes uj felhasznalonal / forditasnál egygel több jön létre ezekből.

A két "tábla", vagyis kollekcio a username oszlopnevekkel vannak összeköttetve.

3.5. Fejlesztési leírás

3.5.1. Fejlesztési folyamat

Fejlesztés előtti felkészülés:

- Biztosítsuk a szükséges .env konfigurációt

- Telepítsük az npm csomagokat: `npm install`
- Indítsuk el az dev-és backend szervereket

a) **Új komponens hozzáadása:**

- Hozzunk létre egy új JSX fájlt a `src/components` mappában
- Módosítsuk
- Adjuk hozzá az `App.jsx`-hez a route-ot

b) **Stílusozás:**

- SCSS fájlokat a komponenssel azonos névvel hozzunk létre
- változókat a `'variables.scss'` fájlban tartsuk
- Sötét mód: `'[data-theme="dark"]selector`

3.5.2. Tesztelés

Jest-et használunk:

```
npm test
npm run test:watch
npm run test:coverage
```

- Jest konfigurációt a `jest.config` fájlban,
- unit teszteket a `components/tests` és `tests` mappában találhatjuk.

```
anon@artix: [~/Projects/Websites/nudli/tests/full-react-app]
$ npm test

> nudli-fordito@0.0.1 test
> jest

PASS  __tests__/auth.test.js
PASS  src/components/__tests__/TranslateText.test.jsx
PASS  src/components/__tests__/MongoConnect.test.jsx

Test Suites: 3 passed, 3 total
Tests:       22 passed, 22 total
Snapshots:   0 total
Time:        4.414 s
Ran all test suites.
```

24. ábra. Sikeres tesztek regisztrációnál, bejelentkezésnél és fordításnál

```
Test Suites: 1 failed, 3 passed, 4 total
Tests:       1 failed, 22 passed, 23 total
Snapshots:   0 total
Time:        3.566 s, estimated 4 s
Ran all test suites.
```

25. ábra. Ha valamiféle hiba történne teszteléskor, ezt látnánk.

3.5.3. Build és deployment

Ahhoz, hogy projektünket egy kész weboldalra fordítsuk, adjuk ki a következő parancsot:

```
npm run build
```

Ennek hatására létrejön egy *dist/* mappa, melyben megtalálható az összes .js, html, CSS fájl. Ezek a Javascript fájlok bundled formává alakítódnak, amely lényegében azt jelenti, hogy összetömörítve / becsomagolva kapjuk meg.

Ha a dist/ mappából el szeretnénk indítani a weboldalt, írjuk be a lefordított oldal elindításának parancsát:

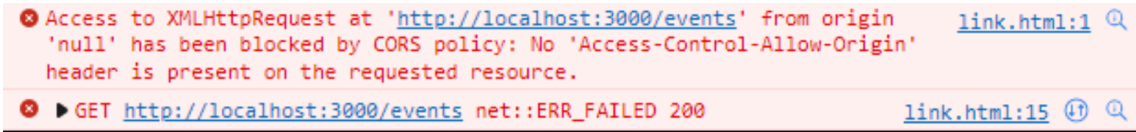
```
npm run start
```

3.6. Hibakezelés és megoldások

3.6.1. Általunk észlelt problémák ezelőtt

CORS hiba:

- **Probléma:** F12 konzol hibát jelez: a frontend nem tud csatlakozni a backend-hez
- **Megoldás:** Ellenőrizzük, hogy a CORS beállítások valóban helyesek, mivel ez engedélyezi a két port közötti (5713 és 3001) kommunikációt



26. ábra. Az említett CORS hiba

MongoDB szinkronizációs hiba:

- **Probléma:** Bejelentkezés, regisztráció nem működik
- **Megoldás:** Ellenőrizzük az MONGO_URI változót, valamint a Cluster állapotát (nem-e töröltük az adatbázist? nincs-e IP címünk tiltva?)

Felhasználónév

teszteles

Jelszó

...

Mutat

Belépés

Hálózati hiba

27. ábra. MongoDB "Hálózati hiba"

DeepL API Limitálás:

- **Probléma:** "Quota exceeded" hibaüzenet

- **Magyarázat:** A DeepL API-nak az ingyenes csomagja csak fél-millió karakterre limitált havonta. Ha fejlesztőként ebből kifutsz, regisztrálj új fiókot, és igényelj új API kulcsot.

3.7. Jó, ha tudod...

1. **Komponensek elkülönítése:** Minden komponens saját fájlban legyen,
2. **State management:** React useState hookokat használunk
3. **Környezeti változók:** Érzékeny adatok .env-ben
4. **Biztonság:** Jelszavakat bcrypt-el kellene hashelni

3.8. Jövőbeli fejlesztési lehetőségek (roadmap)

1. **API függetlenség:** Helyi LLM integrálása
2. **Haladó fordítási előzmények:** Rendezés, keresés
3. **Mobilalkalmazás:** Expo használata, habár ehhez az alkalmazás kódbázisának (úgy 50%) újraírása szükséges
4. **Szóincs kezelés:** Fordítási szótár - szó/kifejezés szótár a fordítási konzisztencia érdekében
5. **Fordítási memória:** TM integrálás (Translation Memory) az ismétlődő tartalmakhoz

4. Dokumentáció elkészítése

A dokumentációt Béla készítette. Dokumentáció készítés legelső változatakor 11ty (eleventy), egy statikus oldalgenerátorral írtuk a dokumentációt, azonban technikai okok miatt és letisztultság érdekében nyomtatható PDF formátumra váltottunk.

További egyszerűség és átláthatóság érdekében a dokumentáció még mindig Markup nyelvben íródik (nevét illetően: Emacs Org), azonban ezt $\text{\LaTeX}$ kóddá, majd dokumentummá alakítva publikáljuk.

Céлом egy szép és átlátható dokumentum megírása volt. Mivel már évek óta nagy Org Mode és $\text{\LaTeX}$ rajongó vagyok, valamint tucatnyi tudományos cikket, így ideillő, hogy a technikumi projektünk dokumentációja is $\text{\LaTeX}$ -ben íródjon. Alább részletezem ennek a folyamatát, továbbá egy kis háttér információval illetem.

4.1. Mi az az Emacs Org Mode?

Az Emacs Org Mode valójában a GNU Emacs szövegszerkesztő programnak egy (2006 óta) beépített jegyzetelő rendszere. Az Org jegyzetek jelentősen eltérnek Markdown jegyzetektől szintaktikailag, de egy frissen kezdő számára is néhány perc után könnyen elsajátíthatóvá válik. Az Org Mode-ot *Carsten Dominik* készítette 2003-ban a GNU Emacs-hez, majd számos hozzájárulás és pár év várakozás után 2006-ban alapértelmezetten adták ki felhasználóknak.



28. ábra. Emacs logója



29. ábra. Org Mode logója, a következő felirattal: "**Szervezd** az életed, *sim*a szövegben"

Egyszerű szintaxis, jól olvasható nyers szövegben is. Tud kódrészlet-futtatást, táblázatkezelést, naptári események felírását. Talán amiben legjobban fénylik, az az exportálás különböző formátumokba (PDF, HTML, $\text{\LaTeX}$ stb.), és az ebből következő matematikai egyenletek renderelése.

A GitHub tökéletesen megjeleníti az Org jegyzeketet, csakúgy mint a Markdown jegyzeteket.

4.2. $\text{\LaTeX}$, avagy Latex

A $\text{\LaTeX}$ egy szövegszerkesztő-typesetter rendszer, amit főleg tudományos és műszaki dokumentumok írására használnak. Nem úgy működik, mint egy hagyományos szövegszerkesztő—Word, LibreOffice—, ahol mindent kézzel formázunk, hanem parancsokkal alakítjuk a dokumentum kimenetelét, és a tördelést a rendszer végzi el. Az, aki ért a $\text{\LaTeX}$ nyelvén, hatékonyan és gyorsan képes dokumentumokat írni.

Erőssége a matematikai képletek kezelése, a hivatkozások és az automatikus számozás. Nagyobb dokumentumoknál (szakdolgozat, cikk, könyv) különösen jól jön, mert a struktúra tiszta marad, és nem esik szét a formázás. Éppen ezért döntöttünk úgy, hogy ebben a formátumban fogunk dolgozni.

Technikai információ: a *WYSIWYG* (What You See Is What You Get) egy kifejezés, mely azokra a programcsomagokra, szoftverre utal, melyeken a felhasználó olyan funkciókat használ, ahol az adott funkció vizuálisan megjelenik. Például Word-ben egy mondat félkövérré alakítását kijelöléssel, majd a félkövér "B" gombra kattintva tehetjük. Azonban $\text{\LaTeX}$ egy *YAFIYGI* (You Asked For It, You Got It) szoftver, mely az előbbi ellentéte. $\text{\LaTeX}$ -ban csupán parancsokkal alakítjuk a dokumentumunkat, kivéve ha egy *WYSIWYG* környezetet használunk, mint például az *Overleaf* vagy *LyX*.

Rövid példa:

```
\documentclass{article}
\begin{document}
\section{Bevezetés}
\[
E = mc^2
\]
\end{document}
\end{lstlisting}
```

a következő grafikát jeleníti meg:

$$E = mc^2$$

4.3. Org Mode és $\text{\LaTeX}$ együtt használva

Az Org Mode és a $\text{\LaTeX}$ jól kiegészítik egymást. Elvégre az Org Mode részben erre is lett kialakítva. Org-ban megírhatjuk a dokumentumot egyszerű, áttekinthető formában, majd $\text{\LaTeX}$ -be exportáljuk, és egy szép, letisztult PDF-et kapunk.

Például Org-ban így néz ki egy részlet:

Ez egy képlet:
 $\backslash (E = mc^2 \backslash)$

Amely PDF exportálásnál így néz ki:

Ez egy képlet: $E = mc^2$

Innen egy paranccsal lehet $\text{\LaTeX}$ -be, majd PDF-be exportálni. Vagy külön $\text{\LaTeX}$ -ba. Teljesen rajtunk múlik! Ez a parancs a háttérben egy saját .org -> .tex fordítót futtat, majd egy latexmk parancsot futtat, ami .tex fájlból .pdf fájlt alakít.

Lehet közvetlen $\text{\LaTeX}$ -blokkokat is írni Org-ban:

```
#+begin_export latex
\begin{equation}
E = mc^2
\end{equation}
#+end_export
```

Így a szerkesztés egyszerű marad. **Gyakorlatban ez azt jelenti, hogy nem a formázással kell küzdenünk, hanem foglalkozhatunk szabadon a tartalommal.**